

מערכות הפעלה – תרגיל 3

לתרגיל זה 2 חלקים:

- א. פיתוח רשימה מקושרת הבטוחה לשימוש במקביל.
 - ב. כתיבת מודול גרעין להחלפת קריאת מערכת לבחירת המשתמש.
- בסוף התרגיל מפורטות הוראות ההגשה והנחיות כלליות. יש לקרוא אותן בעיון ולהקפיד על כל ההנחיות!

חלק א

עליכם לממש רשימה מקושרת בקובץ C בשם `mylist.c`.

הרשימה תמומש באופן הקלאסי עם הקצאה דינמית (מצביעי `next, prev`), כאשר לכל איבר ברשימה יש ערך **מטיפוס `int`**. לכל רשימה יש גודל מקסימלי הנקבע באתחול.

בנוסף, הרשימה תתמוך במקביליות כך שכל פעולה תעבוד באופן נכון ובטוח גם כאשר מספר חוטים ניגשים לרשימה בו-זמנית. כדי לתמוך במקביליות יש לממש את הרשימה באמצעות **מנעול ומשתני תנאי**.

הרשימה עצמה תוגדר בתור `struct` בשם `MyList`, ותתמוך בפעולות הבאות:

- **אתחול רשימה:** מקבלת גודל מקסימלי ומאתחלת רשימה חדשה. ניתן להניח שפונקציה זו נקראת פעם אחת בלבד, לפני השימוש ברשימה ולא במקביל.
- **הריסת רשימה:** מוחקת רשימה ע"י שחרור כל ההקצאות הרלוונטיות (כולל האיברים שנשארו ברשימה). ניתן להניח שפונקציה זו נקראת פעם אחת בלבד, אחרי כל שימוש ברשימה ולא במקביל.
- **גודל רשימה:** מחזירה את כמות האיברים ברשימה.
- **הוספה לראש:** מקבלת ערך מטיפוס `int` ומוסיפה אותו לראש הרשימה. אם הרשימה בגודל המקסימלי, יש לחסום את החוט הקורא עד שמתאפשר להוסיף את הערך.
- **הוספה לזנב:** מקבלת ערך מטיפוס `int` ומוסיפה אותו בסוף הרשימה. אם הרשימה בגודל המקסימלי, יש לחסום את החוט הקורא עד שמתאפשר להוסיף את הערך.
- **הסרה מהראש:** מסירה את האיבר הראשון (בראש הרשימה) ומחזירה את ערכו. אם הרשימה ריקה, יש לחסום את החוט הקורא עד שמתאפשר להסיר את הערך.
- **הסרה מהזנב:** מסירה את האיבר האחרון (בסוף הרשימה) ומחזירה את ערכו. אם הרשימה ריקה, יש לחסום את החוט הקורא עד שמתאפשר להסיר את הערך.

שלד של הגדרת הרשימה והצהרת הפונקציות לכל פעולה מצורף לתרגיל בקובץ `mylist.c`. אין לשנות את ההצהרות והחתימות בקובץ בשום צורה. ניתן ורצוי להוסיף פונקציות עזר והגדרות נוספות לצורך המימוש.

שימו לב:

- אין לממש את הרשימה באמצעות מערך אלא כרשימת מצביעים בלבד (המימוש הקלאסי).
- יש להימנע מהגדרת משתנים גלובליים. הפתרון צריך לתמוך במספר רשימות, כאשר בכל אחת ניתן להשתמש באופן נפרד ובמקביל לרשימות אחרות. לכל רשימה יכול להיות ערך מקסימום שונה.
- יש להגיש את הקוד **ללא `main`**. הוספת `main` לקוד תגרום לכישלון הבדיקה ופסילה של כל החלק.
- במימוש התוכנית יש להשתמש במנעולים ומשתנאי תנאי **בלבד** ולא בפונקציות ספריה של שפת C, `malloc`, `free`. יש לבדוק את ערכי ההחזר של **כל קריאות המערכת** (כולל `malloc`).

מערכות הפעלה – תרגיל 3

חלק ב

עליכם לממש מודול גרעין בקובץ C בשם `mymod.c`, אשר מחליף את אחת מקריאות המערכת הבאות: `mkdir`, `chdir`, `close`, `dup`.

המודול מקבל 2 פרמטרים: פרמטר ראשון בשם `sysnr` מסוג מספר (`int`) המציין את מספר קריאת המערכת שיש להחליף, ופרמטר שני בשם `msg` מסוג מחרוזת.

מספרי קריאות המערכת נתונים ע"י הקבועים `__NR_xyz` (את `xyz` יש להחליף בשם הקריאה). במידה והתקבל מספר לא תקין (שאינו תואם לאחת הקריאות) או שלא הוזן מספר, טעינת המודול תיכשל.

בכל פעם שנקראת הקריאה שהוחלפה, יש להדפיס הודעה ואז להפעיל את קריאת המערכת המקורית. את ההודעה יש להדפיס לפי הפרמטר שהתקבל, **לפני** ביצוע הקריאה עצמה (קריאת המערכת המקורית):

- עבור `mkdir`, `chdir` יש לשרשר להודעה את שם התיקייה עליה בוצעה הקריאה (עד 15 תווים).
- עבור `close`, `dup` יש לשרשר להודעה את מספר ה-FD עליו בוצעה הקריאה.

במחיקת המודול יש לדאוג לשחזר את הכל למצב הקודם ולהמתין **3 שניות** לפני הסיום.

החתימות של קריאות מערכת אלו בקרנל הן:

```
asmlinkage long chdir(const char __user *pathname)
asmlinkage long mkdir(const char __user *pathname, umode_t mode)
asmlinkage long close(unsigned int fd)
asmlinkage long dup(unsigned int fildes)
```

יחד עם מודול הגרעין יש לצרף גם קובץ Makefile מתאים אשר מקמפל אותו, וקובץ סקריפט בשם `mymod.sh` אשר מקבל ארגומנט יחיד ומבצע את הפעולות הבאות (לפי הסדר):

- מעתיק את קובץ הקוד של המודול יחד עם ה-Makefile אל התיקייה הנתונה בארגומנט הראשון.
- מקמפל את המודול וטוען אותו אל מערכת ההפעלה.
- מוחק את כל הקבצים הזמניים שנוצרו: `sudo make clean`.
- מוחק את 2 הקבצים שהועתקו אל תיקיית היעד (את העותקים, הקבצים המקוריים לא נמחקים).

ניתן להניח שמתקבל ארגומנט יחיד והארגומנט תקין, ואת קובץ הסקריפט מריצים עם `sudo`.

מערכות הפעלה – תרגיל 3

הוראות הגשה:

- יש להגיש את התרגיל בקובץ tar.gz **יחיד**. שם הקובץ הוא ת"ז בלבד (9 ספרות) וסיומת. לדוגמה, אם הת"ז הוא 123456789 אז שם הקובץ להגשה יהיה **123456789.tar.gz**
- הקובץ יכול את 4 קבצי התרגיל **בלבד**, ללא תיקיות או קבצים נסתרים. את הקובץ יש ליצור ב-VM באמצעות הפקודה הבאה:
tar czf <yourid>.tar.gz mylist.c mymod.c Makefile mymod.sh
- מצורף לתרגיל סקריפט פייתון (בשם checksub.py). יש להריץ אותו עם שם קובץ ההגשה כפרמטר, לבדיקת תקינות ההגשה. **לא יתקבלו ערעורים על הגשה לא תקינה.**

הנחיות כלליות:

- ההגשה היא **ביחידים**. כל דמיון בין הגשות **ולו הקטן ביותר** יועבר מיידית לטיפול ועדת המשמעת של המכללה ללא אזהרה או אפשרות ערעור. מקורות אונליין (כולל LLM כמו ChatGPT) שקולים להעתקה. פתרו את התרגיל **באופן עצמאי לחלוטין** והימנעו מבעיות.
- הציון על שאלת הקוד מתמקד בנכונות הקוד (קומפילציה ללא אזהרות, בדיקה וטיפול בשגיאות, ניהול משאבים וזיכרון, וכו') ולא עיצוב הקוד (תיעוד, חלוקה לפונקציות, וכו'). עם זאת, יש לכתוב קוד קריא וברור. הקוד יקומפל באמצעות הפקודה:
gcc <codefiles> -Wall -pthread -o <exefile>
- בכל ערעור על קוד יש לצרף תמונת מסך של הטרמינל עם הפלט של פקודת **hostnamectl**, ביצוע **gcc** לקוד, והרצה של הקוד עם פלט תקין – מתוך VM על גבי **vagrant** כפי שנלמד. מי שלא פועל לפי ההנחיות (מערכת הפעלה שונה, או ללא **vagrant**) – **מוותר על הזכות לערער**.
- **הגשות באיחור** יתקבלו עם הפחתה של 10 נק' לכל יום איחור מעבר למועד ההגשה, אשר מסתיים בדיוק **בשעה 23:00**. דאגו להגיש מספיק זמן מראש, לא יתקבל אישור על פספוס ברגע האחרון. במקרה של מחלה, מילואים, או כל סיבה מוצדקת אחרת, יש לשלוח אישור אל המרצה **לפחות 24 שעות** לפני ההגשה.